

Department of Computer Science and Engineering

Assignment Questions

Course Code & Name : CSA0921 – Programming in Java

Assignment Title:	Smart Campus Resource Allocation and Facility Management System using Java
Course Outcome(s) – CO:	CO1: Apply object-oriented programming principles such as classes, objects, encapsulation, data hiding, inheritance, and polymorphism to design and implement entity manipulations (BL3,BL4) CO2: Implement Collection Framework interfaces such as List, ArrayList, Set, Map, HashTable, and HashMap using iterators and generics to store, retrieve, and process groups of objects in web applications.(BL3,BL4) CO3: Build multithreaded Java programs by applying thread priorities, synchronisation, and inter-thread communication mechanisms to achieve concurrent program execution and use exception handling techniques (BL3,BL4)
Bloom's Taxonomy Level	L3 – Apply; L4 – Analyse
SDG Mapping (SDG 1-17)	SDG 9 – Industry, Innovation and Infrastructure SDG 4 – Quality Education SDG 11 – Sustainable Cities and Communities

Problem Statement : A large autonomous university manages shared facilities such as computer laboratories, seminar halls, meeting rooms, multimedia studios, and innovation centers. Students, faculty, and researchers submit reservation requests for classes, workshops, examinations, and project activities. The current manual booking system often leads to scheduling conflicts, duplicate reservations, and inefficient utilization of campus resources. The University ICT Cell plans to develop a Java-based Smart Campus Resource Allocation and Facility

Management System to automate the reservation process. The application should maintain user profiles, manage facility details, process bookings, modify or cancel reservations, and generate utilization reports for administrators. The system must apply Object-Oriented Programming principles by modeling users and facilities through classes, inheritance, encapsulation, and polymorphism. Resource records and booking history should be managed using the Java Collection Framework with generics, lists, maps, and iterators. To support multiple users accessing the system simultaneously, the application should implement multithreading, synchronization, inter-thread communication, and appropriate exception handling. The final solution should be a menu-driven application that ensures efficient, reliable, and conflict-free campus resource management.

Assessment Rubrics

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
1. Requirement Analysis, Java Fundamentals & Program Logic	15	Clearly understands all requirements, entities, workflows, and user interactions; uses appropriate data types, operators, conditional statements, loops, and decision-making structures efficiently and correctly.	Most requirements and workflows are identified; Java fundamentals and control structures are used correctly with minor errors.	Basic requirements are identified; fundamental constructs are used with some logical errors or inefficiencies.	Requirements are poorly understood; frequent errors in Java fundamentals and program logic affect correctness.
2. Class Design, Encapsulation, Inheritance & Polymorphism	20	Well-structured classes for users, facilities, bookings, and management; strong encapsulation and data hiding;	Classes are reasonably designed with appropriate encapsulation and inheritance; polymorphism	Basic classes and inheritance are present, but encapsulation or polymorphic behavior is incomplete.	Poor class organization; weak encapsulation; inheritance or polymorphism is absent or incorrectly

		appropriate inheritance, method overriding, and runtime polymorphism are implemented effectively.	is implemented with minor limitations.		implemented.
3. Java Collection Framework, Generics & Data Management	20	Appropriate use of List, ArrayList, Set, HashSet, Map, and HashMap with generics; iterators are effectively used for storing, searching, updating, and processing campus resources and booking records.	Appropriate collections and generics are used with minor implementation or retrieval issues.	Collections are used for basic storage and retrieval, but some data structures or iterator operations are inappropriate or incomplete.	Collections, generics, or iterators are largely missing, incorrectly implemented, or inefficiently used.
4. Multithreading, Synchronization, Exception Handling & Menu-Driven Functionality	25	Multiple threads effectively handle concurrent booking, cancellation, and facility monitoring; synchronization and inter-thread communication prevent race conditions; comprehensive exception handling manages invalid inputs, duplicate bookings, unavailable facilities, and scheduling conflicts; all menu operations function reliably.	Multithreading, synchronization, exception handling, and menu operations are mostly implemented correctly with minor concurrency or runtime issues.	Basic multithreading and exception handling are present, but synchronization, communication mechanisms, or some menu functions are incomplete.	Multithreading or synchronization is absent/non-functional; exception handling is inadequate and major menu operations are missing or unreliable.

5. System Integration, Reports, Documentation & Overall Quality	10	All modules are fully integrated; facility availability, booking statistics, utilization reports, and other required outputs are generated correctly; code is organized, readable, documented, and professionally presented.	Modules are well integrated and reports are mostly correct; code organization and documentation are satisfactory.	Basic integration and reporting are achieved, but some reports, documentation, or code quality need improvement.	Poor integration; reports are missing or incorrect; code is difficult to understand and documentation is inadequate.
6. Reflection: Design Decisions, SDG Relevance & Learning Outcomes	10	Provides thoughtful justification of design decisions; clearly connects the system with relevant SDGs such as SDG 4, SDG 9, and SDG 11 ; provides specific reflection on challenges, problem-solving, and learning outcomes.	Provides reasonable justification of design choices and SDG relevance; discusses challenges and learning with adequate detail.	Reflection is present but generic; limited justification of design decisions, SDG relevance, or learning outcomes.	Reflection is missing or lacks meaningful discussion of design decisions, SDG relevance, challenges, or learning outcomes.

Deliverables

To receive full credit, each submission must include the following deliverables:

1. Pseudo code
2. Implementation & Results
3. Github Upload

CODE:

```
import java.time.LocalDateTime;

import java.time.format.DateTimeFormatter;

import java.util.*;

import java.util.concurrent.atomic.AtomicInteger;


public class SmartCampusSystem {

    static final DateTimeFormatter FMT =
        DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm");

    // ----- USER CLASS -----

    static abstract class User {

        private final String id;

        private final String name;

        private final String email;

        User(String id, String name, String email) {

            this.id = id;

            this.name = name;

            this.email = email;

        }

    }

}
```

```
public String getId() {  
    return id;  
}
```

```
public String getName() {  
    return name;  
}
```

```
public String getEmail() {  
    return email;  
}
```

```
public abstract String getRole();  
}
```

```
// ----- INHERITANCE -----
```

```
static class Student extends User {  
    Student(String id, String name, String email) {  
        super(id, name, email);  
    }  
}
```

```
public String getRole() {  
    return "Student";  
}  
}
```

```
static class Faculty extends User {  
    Faculty(String id, String name, String email) {  
        super(id, name, email);  
    }  
}
```

```
public String getRole() {  
    return "Faculty";  
}  
}
```

```
static class Researcher extends User {  
    Researcher(String id, String name, String email) {  
        super(id, name, email);  
    }  
}
```

```
public String getRole() {  
    return "Researcher";  
}
```

```
    }  
}  
  
// ----- FACILITY -----  
  
static class Facility {  
    private final String id;  
    private final String name;  
    private final String type;  
    private final int capacity;  
    private boolean active = true;  
  
    Facility(String id, String name, String type, int capacity) {  
        this.id = id;  
        this.name = name;  
        this.type = type;  
        this.capacity = capacity;  
    }  
  
    String getId() {  
        return id;  
    }  
}
```



```
String getName() {  
    return name;  
}
```

```
String getType() {  
    return type;  
}
```

```
int getCapacity() {  
    return capacity;  
}
```

```
boolean isActive() {  
    return active;  
}  
}
```

```
// ----- BOOKING -----
```

```
static class Booking {  
    private final String id;  
    private final String userId;  
    private final String facilityId;
```

```
private LocalDateTime start;  
private LocalDateTime end;  
private String purpose;  
private String status;
```

```
Booking(String id, String userId, String facilityId,  
        LocalDateTime start, LocalDateTime end,  
        String purpose) {
```

```
    this.id = id;  
    this.userId = userId;  
    this.facilityId = facilityId;  
    this.start = start;  
    this.end = end;  
    this.purpose = purpose;  
    this.status = "CONFIRMED";  
}
```

```
String getId() {  
    return id;  
}
```

```
String getUserId() {  
    return userId;  
}
```

```
String getFacilityId() {  
    return facilityId;  
}
```

```
LocalDateTime getStart() {  
    return start;  
}
```

```
LocalDateTime getEnd() {  
    return end;  
}
```

```
String getStatus() {  
    return status;  
}
```

```
void modify(LocalDateTime start,
```

```

        LocalDateTime end,

        String purpose) {

    this.start = start;

    this.end = end;

    this.purpose = purpose;
}

void cancel() {
    status = "CANCELLED";
}
}

// ----- CAMPUS MANAGER -----

static class CampusManager {

    // List and ArrayList

    private final List<User> users =

        new ArrayList<User>();

    private final List<Facility> facilities =

        new ArrayList<Facility>();

```

```
// Set and HashSet
```

```
private final Set<String> facilityIds =  
    new HashSet<String>();
```

```
// Map and HashMap
```

```
private final Map<String, User> userMap =  
    new HashMap<String, User>();
```

```
private final Map<String, Booking> bookingMap =  
    new HashMap<String, Booking>();
```

```
private final Map<String, List<Booking>> facilityBookings =  
    new HashMap<String, List<Booking>>();
```

```
// Hashtable
```

```
private final Hashtable<String, String> facilityTypeTable =  
    new Hashtable<String, String>();
```

```
private final AtomicInteger bookingSeq =  
    new AtomicInteger(1000);
```

```
// ----- ADD USER -----  
  
synchronized void addUser(User user) throws Exception {  
  
    if (user == null ||  
        userMap.containsKey(user.getId())) {  
  
        throw new Exception("Duplicate user ID.");  
    }  
  
    users.add(user);  
    userMap.put(user.getId(), user);  
}  
  
// ----- ADD FACILITY -----  
  
synchronized void addFacility(Facility facility)  
    throws Exception {  
  
    if (facility == null ||  
        facilityIds.contains(facility.getId())) {  
  
        throw new Exception("Duplicate facility ID.");  
    }  
}
```

```
facilities.add(facility);  
facilityIds.add(facility.getId());
```

```
facilityBookings.put(  
    facility.getId(),  
    new ArrayList<Booking>());
```

```
facilityTypeTable.put(  
    facility.getId(),  
    facility.getType());  
}
```

```
// ----- FIND USER -----
```

```
User getUser(String id) {  
    return userMap.get(id);  
}
```

```
// ----- FIND FACILITY -----
```

```
Facility getFacility(String id) {  
  
    for (Facility f : facilities) {
```

```

        if (f.getId().equals(id)) {
            return f;
        }
    }

    return null;
}

// ----- CREATE BOOKING -----
synchronized Booking createBooking(
    String userId,
    String facilityId,
    LocalDateTime start,
    LocalDateTime end,
    String purpose) throws Exception {

    if (getUser(userId) == null) {
        throw new Exception("User not found.");
    }

    Facility facility =

```



```
getFacility(facilityId);
```

```
if (facility == null ||  
    !facility.isActive()) {  
  
    throw new Exception(  
        "Facility unavailable.");  
}
```

```
if (start == null ||  
    end == null ||  
    !start.isBefore(end)) {  
  
    throw new Exception(  
        "Invalid time range.");  
}
```

```
List<Booking> list =  
    facilityBookings.get(facilityId);
```

```
// Conflict checking  
for (Booking b : list) {
```

```
        if ("CONFIRMED".equals(b.getStatus()) &&  
            start.isBefore(b.getEnd()) &&  
            end.isAfter(b.getStart())) {  
  
            throw new Exception(  
                "Scheduling conflict.");  
        }  
    }  
}
```

```
Booking booking =  
    new Booking(  
        "B" + bookingSeq.getAndIncrement(),  
        userId,  
        facilityId,  
        start,  
        end,  
        purpose);
```

```
bookingMap.put(  
    booking.getId(),  
    booking);
```

```
list.add(booking);

return booking;
}

// ----- MODIFY BOOKING -----
synchronized void modifyBooking(
    String bookingId,
    LocalDateTime start,
    LocalDateTime end,
    String purpose) throws Exception {

    Booking old =
        bookingMap.get(bookingId);

    if (old == null) {
        throw new Exception(
            "Booking not found.");
    }

    if (!"CONFIRMED".equals(old.getStatus())) {
```

```

        throw new Exception(
            "Booking is cancelled.");
    }

    for (Booking b :
        facilityBookings.get(
            old.getFacilityId())) {

        if (!b.getId().equals(bookingId) &&
            "CONFIRMED".equals(b.getStatus()) &&
            start.isBefore(b.getEnd()) &&
            end.isAfter(b.getStart())) {

            throw new Exception(
                "Scheduling conflict.");
        }
    }

    old.modify(start, end, purpose);
}

// ----- CANCEL BOOKING -----

```

```

synchronized void cancelBooking(
    String bookingId) throws Exception {

    Booking booking =
        bookingMap.get(bookingId);

    if (booking == null) {
        throw new Exception(
            "Booking not found.");
    }

    booking.cancel();
}

// ----- LIST USERS -----

void listUsers() {

    System.out.println("\n--- USERS ---");

    System.out.printf(
        "%-6s %-15s %-25s %-12s%n",
        "ID", "Name", "Email", "Role");
}

```

```

        Iterator<User> iterator =
            users.iterator();

        while (iterator.hasNext()) {

            User user = iterator.next();

            System.out.printf(
                "%-6s %-15s %-25s %-12s%n",
                user.getId(),
                user.getName(),
                user.getEmail(),
                user.getRole());
        }
    }

    // ----- LIST FACILITIES -----

    void listFacilities() {

        System.out.println(
            "\n--- FACILITIES ---");
    }

```

```
System.out.printf(
    "%-6s %-20s %-20s %-8s %-10s%n",
    "ID",
    "Name",
    "Type",
    "Capacity",
    "Status");
```

```
Iterator<Facility> iterator =
    facilities.iterator();
```

```
while (iterator.hasNext()) {
```

```
    Facility f = iterator.next();
```

```
System.out.printf(
    "%-6s %-20s %-20s %-8d %-10s%n",
    f.getId(),
    f.getName(),
    f.getType(),
    f.getCapacity(),
```

```

        f.isActive()
            ? "Available"
            : "Inactive");
    }
}

// ----- LIST BOOKINGS -----

void listBookings() {

    System.out.println(
        "\n--- ALL BOOKINGS ---");

    if (bookingMap.isEmpty()) {

        System.out.println(
            "No bookings.");

        return;
    }

    for (Booking b :
        bookingMap.values()) {

```



```

        System.out.println(
            b.getId()
            + " | User="
            + b.getUserId()
            + " | Facility="
            + b.getFacilityId()
            + " | "
            + b.getStart().format(FMT)
            + " to "
            + b.getEnd().format(FMT)
            + " | "
            + b.getStatus());
    }
}

// ----- USER BOOKINGS -----

void userBookings(String userId) {

    System.out.println(
        "\n--- USER BOOKINGS ---");

```

```

for (Booking b :
    bookingMap.values()) {

    if (b.getUserId().equals(userId)) {

        System.out.println(
            b.getId()
            + " | "
            + b.getFacilityId()
            + " | "
            + b.getStart().format(FMT)
            + " to "
            + b.getEnd().format(FMT)
            + " | "
            + b.getStatus());
    }
}

// ----- SEARCH FACILITY -----

void searchFacility(String text) {

```

```
System.out.println(
    "\n--- SEARCH RESULTS ---");

for (Facility f : facilities) {

    if (f.getName()
        .toLowerCase()
        .contains(text.toLowerCase())
        ||
        f.getType()
        .toLowerCase()
        .contains(text.toLowerCase())) {

        System.out.println(
            f.getId()
            + " - "
            + f.getName()
            + " ("
            + f.getType()
            + ")");
    }
}
```

```
}
```

```
// ----- UTILIZATION REPORT -----
```

```
void utilizationReport() {
```

```
    System.out.println(
```

```
        "\n--- UTILIZATION REPORT ---");
```

```
    System.out.printf(
```

```
        "%-20s %-8s %-12s %-15s\n",
```

```
        "Facility",
```

```
        "Bookings",
```

```
        "Confirmed",
```

```
        "Cancelled");
```

```
    for (Facility f : facilities) {
```

```
        int total = 0;
```

```
        int confirmed = 0;
```

```
        int cancelled = 0;
```

```
        for (Booking b :
```

```
        facilityBookings.get(
            f.getId())) {

    total++;

    if ("CONFIRMED".equals(
        b.getStatus())) {

        confirmed++;

    } else {

        cancelled++;

    }

}

System.out.printf(
    "%-20s %-8d %-12d %-15d%n",
    f.getName(),
    total,
    confirmed,
    cancelled);
```

```

    }
}

// ----- MULTITHREADING -----

void concurrentDemo()
    throws Exception {

    System.out.println(
        "\n--- MULTITHREADING DEMO ---");

    LocalDateTime start =
        LocalDateTime.of(
            2026, 9, 20, 10, 0);

    LocalDateTime end =
        LocalDateTime.of(
            2026, 9, 20, 12, 0);

    Runnable task = new Runnable() {

        public void run() {

```

```
String userId;

if (Thread.currentThread()
    .getName()
    .equals("Thread-1")) {

    userId = "S101";

} else {

    userId = "F201";
}

try {

    Booking b =
        createBooking(
            userId,
            "F103",
            start,
            end,
            "Concurrent Test");
```

```
        System.out.println(
            Thread.currentThread()
                .getName()
            + " -> SUCCESS: "
            + b.getId());

    } catch (Exception e) {

        System.out.println(
            Thread.currentThread()
                .getName()
            + " -> BLOCKED: "
            + e.getMessage());
    }
}

};
```

```
Thread t1 =
    new Thread(task, "Thread-1");
```

```
Thread t2 =
```



```
        new Thread(task, "Thread-2");

// Thread priorities

t1.setPriority(
    Thread.MAX_PRIORITY);

t2.setPriority(
    Thread.NORM_PRIORITY);

t1.start();
t2.start();

t1.join();
t2.join();

System.out.println(
    "Both threads completed safely.");
}

// ----- SAMPLE DATA -----

void seed() throws Exception {
```

```
addUser(  
    new Student(  
        "S101",  
        "Ananya",  
        "ananya@campus.edu"));
```

```
addUser(  
    new Faculty(  
        "F201",  
        "Dr. Kumar",  
        "kumar@campus.edu"));
```

```
addUser(  
    new Researcher(  
        "R301",  
        "Rahul",  
        "rahul@campus.edu"));
```

```
addFacility(  
    new Facility(  
        "F101",  
        "Computer Lab 1",
```

```
        "Computer Laboratory",  
        60));
```

```
addFacility(  
    new Facility(  
        "F102",  
        "Seminar Hall A",  
        "Seminar Hall",  
        120));
```

```
addFacility(  
    new Facility(  
        "F103",  
        "Multimedia Studio",  
        "Multimedia Studio",  
        40));
```

```
addFacility(  
    new Facility(  
        "F104",  
        "Innovation Center",  
        "Innovation Center",
```

```

            80));

        }

    }

// ----- DATE INPUT -----

static LocalDateTime readDate(

    Scanner sc,

    String prompt) {

    while (true) {

        try {

            System.out.print(prompt);

            return LocalDateTime.parse(

                sc.nextLine().trim(),

                FMT);

        } catch (Exception e) {

            System.out.println(

```

```
        "Invalid date. Use yyyy-MM-dd HH:mm");  
    }  
}  
}
```

```
// ----- PAUSE -----
```

```
static void pause(Scanner sc) {
```

```
    System.out.println(
```

```
        "\nPress Enter to return to menu...");
```

```
    sc.nextLine();
```

```
}
```

```
// ----- MAIN -----
```

```
public static void main(String[] args) {
```

```
    CampusManager manager =
```

```
        new CampusManager();
```

```
    Scanner sc =
```

```
        new Scanner(System.in);
```

```
try {

    manager.seed();

} catch (Exception e) {

    System.out.println(
        e.getMessage());
}

while (true) {

    System.out.println(
        "\n===== SMART CAMPUS RESOURCE ALLOCATION SYSTEM
=====");

    System.out.println(
        "1. List users\n"
        + "2. List facilities\n"
        + "3. Add user\n"
        + "4. Add facility\n")
```

OUTPUT

```
===== SMART CAMPUS RESOURCE ALLOCATION SYSTEM =====
1. List users
2. List facilities
3. Add user
4. Add facility
5. Create booking
6. Modify booking
7. Cancel booking
8. List all bookings
9. Show user's bookings
10. Search facility
11. Utilization report
12. Run multithreading demo
0. Exit
Enter choice: 5
User ID: S101
Facility ID: F101
Start (yyyy-MM-dd HH:mm): 2026-09-10 10:00
End (yyyy-MM-dd HH:mm): 2026-09-10 12:00
Purpose: programmingf
Error: Scheduling conflict.
```

```
===== SMART CAMPUS RESOURCE ALLOCATION SYSTEM =====
1. List users
2. List facilities
3. Add user
4. Add facility
5. Create booking
6. Modify booking
7. Cancel booking
8. List all bookings
9. Show user's bookings
10. Search facility
11. Utilization report
12. Run multithreading demo
0. Exit
Enter choice: 12

--- MULTITHREADING DEMO ---
Thread-2 -> BLOCKED: Scheduling conflict.
Thread-1 -> SUCCESS: B1001
Both threads completed safely.
```

----- SMART CAMPUS RESOURCE ALLOCATION SYSTEM -----

1. List users
2. List facilities
3. Add user
4. Add facility
5. Create booking
6. Modify booking
7. Cancel booking
8. List all bookings
9. Show user's bookings
10. Search facility
11. Utilization report
12. Run multithreading demo
0. Exit

Enter choice: 5

User ID: F201

Facility ID: F101

Start (yyyy-MM-dd HH:mm): 2026-09-10 11:00

End (yyyy-MM-dd HH:mm): 2026_09_10 13:00

Invalid date. Use yyyy-MM-dd HH:mm

End (yyyy-MM-dd HH:mm): Workshop

Invalid date. Use yyyy-MM-dd HH:mm

End (yyyy-MM-dd HH:mm): 2026-09-10 13:00

Purpose: Workshop

Booking confirmed! ID: B1000

3. Add user
4. Add facility
5. Create booking
6. Modify booking
7. Cancel booking
8. List all bookings
9. Show user's bookings
10. Search facility
11. Utilization report
12. Run multithreading demo
0. Exit

Enter choice: 2

--- FACILITIES ---

ID	Name	Type	Capacity	Status
F101	Computer Lab 1	Computer Laboratory	60	Available
F102	Seminar Hall A	Seminar Hall	120	Available
F103	Multimedia Studio	Multimedia Studio	40	Available
F104	Innovation Center	Innovation Center	80	Available

Press Enter to return to menu...

===== SMART CAMPUS RESOURCE ALLOCATION SYSTEM =====

1. List users
2. List facilities
3. Add user
4. Add facility
5. Create booking
6. Modify booking
7. Cancel booking
8. List all bookings
9. Show user's bookings
10. Search facility
11. Utilization report
12. Run multithreading demo
0. Exit

Enter choice: 1

--- USERS ---

ID	Name	Email	Role
S101	Ananya	ananya@campus.edu	Student
F201	Dr. Kumar	kumar@campus.edu	Faculty
R301	Rahul	rahul@campus.edu	Researcher

Press Enter to return to menu...

--- UTILIZATION REPORT ---

Facility	Bookings	Confirmed	Cancelled
Computer Lab 1	1	1	0
Seminar Hall A	0	0	0
Multimedia Studio	1	1	0
Innovation Center	0	0	0